

Meeting 25/03/2026

Summary

Action Items & Next Steps

- Design and implement the Notification & Alerting Microservice (SHIM layer): define the notification body schema, implement `send_notification()` dispatcher, and wire up Slack, Teams, email, and AWS SES delivery channels
- Implement the SHIM layer as both sync and async variants; use a message bus (Kafka container) for async delivery, producer publishes notification events, consumer dispatches to the appropriate channel
- Evaluate OneUptime and OpsGenie as open-source incident management platforms alongside the custom SHIM layer
- Set up kubernetes-based 2-node cluster; organise the repository under `jcrc-system/helms`, `jenkinsfiles`, `scripts_py`, `scripts/` with `apply.sh` entrypoint; explore RKE2 quickstart as an alternative to kubernetes
- Zero-trust security: enforce inter-container mTLS (Istio); define user roles (e.g. `jenkins_admin`) and policies (permissions) and bind them to LDAP groups, no implicit trust between services
- Configure Docker image push pipeline: tag images with the registry path (e.g. `jcrc:ui_college:1.2.1`), push through Nginx reverse proxy to Nexus, with Nexus blob storage backed by MinIO or LocalStack S3
- React frontend: design UI/UX in Figma; implement bearer token auth flow - login form → auth container → bearer token → API requests with auth-token header
- Implement new user KYC registration flow: form submission → Kafka topic → verifier service → verified topic → Users DB → notification queue → welcome message
- Implement existing user login flow: authentication + RBAC → query alumni/fraternity data from DynamoDB or Redis cache
- Implement internal admin user flows: verify/accept/deny new users, add content to the alumni website, create events
- FastAPI REST container for LocalStack: wrap all S3 and DynamoDB operations in FastAPI endpoints; containerise and expose as a service
- Produce overall architecture design for hosting LocalStack on Kubernetes as a Helm chart; coordinate with the infrastructure track
- Define and implement a DynamoDB backup strategy from LocalStack: scheduled export to S3, versioning, and restore procedure

Work Reviewed & Concepts Discussed

Notification & Alerting Microservice — SHIM Layer

- The Notification Microservice is designed as a SHIM (abstraction) layer that decouples the caller from the specific delivery channel.
 - `Send_notification` without knowing whether the message goes to Slack, Teams, email, or a paging service
-

- Notification body schema defined:
 - `message_type`: discriminator field — values are alert, notification, or incident — determines routing priority and delivery urgency
 - `delivery`: array of channel descriptors — each entry specifies the channel (slack, email, teams) and its target (Slack channel name or array, email profile group, Teams webhook)
 - `notification_message`: the human-readable payload sent to all specified channels
- SHIM layer internals: the dispatcher receives the notification body and fans out to the relevant API clients — Slack Web API, Microsoft Teams incoming webhook, SMTP/sendmail, or AWS SES for transactional email
- Sync vs async delivery: synchronous delivery is suitable for low-volume, time-critical alerts; async delivery via a Kafka topic is required for high-volume notifications to prevent the caller from blocking on delivery latency
- Kafka as the message bus: a Kafka container runs as part of the stack; the SHIM producer publishes a notification event to a Kafka topic; the SHIM consumer reads from the topic and dispatches to the channel APIs — decouples producers from delivery speed.

Kubernetes Cluster Setup & Repository Structure

- 2-node Kubernetes cluster provisioned using kubeadm: one control-plane node and one worker node running on VMs, validates that workloads schedule correctly across nodes and that Helm deployments work in a real multi-node environment
- RKE2 (Rancher Kubernetes Engine 2) explored as a production-hardened alternative to kubeadm: ships with containerd, hardened by default (CIS benchmark), and has a simpler install path via the quickstart script evaluated for the next cluster iteration
- Repository layout standardised for the jcrec-system project:
 - `helms/`: Helm charts for each microservice, packaged and versioned
 - `jenkinsfiles/`: declarative pipeline definitions per service
 - `scripts_py/`: Python utility scripts (LocalStack setup, data seeding, backup)
 - `scripts/`: shell scripts with `apply.sh` as the top-level entrypoint for cluster bootstrapping

Zero-Trust Security & IAM Model

- Zero-trust principle applied: no implicit trust between containers every service-to-service call must be authenticated and authorised, regardless of network location
- Istio mTLS: all inter-service traffic inside the mesh is encrypted with mutual TLS; certificates are issued and rotated automatically by Istio's certificate authority (istiod), no manual cert management
- User and role model defined:
 - External users: website visitors unauthenticated public access to alumni and event pages
 - Internal users: website admin, infra admin, infra users authenticated via LDAP, assigned roles
 - Role example: `jenkins_admin` role carries an all-permissions policy; `jenkins_user` role carries a scoped policy roles bound to LDAP groups so membership changes propagate automatically
- Policy enforcement: roles map to fine-grained policies (create, read, update, delete per resource); policies are evaluated at the API gateway and enforced by the auth container before requests reach backend services

Container Image Pipeline & Artifact Storage

- Docker image tagging convention: images are tagged with the full registry path before pushing format: jecrc:<service_name>:<semver> (e.g. jecrc:ui_college:1.2.1) - encodes service identity and version in a single tag
- Push pipeline: docker push routes through the Nginx reverse proxy, which forwards to Sonatype Nexus; Nexus stores the image layers as blobs in its blob store
- Blob storage backend: Nexus blob store configured to use either MinIO (S3 - compatible object storage, self-hosted) or LocalStack S3 abstracts the storage layer so the same Nexus config works locally and in AWS
- LDAP integration with Nexus: Nexus authenticates users against the same LDAP server as Jenkins, a single user directory governs access to both CI and artifact systems
- Helm chart for the full stack: all components (LocalStack, Nexus, Nginx, LDAP, microservices) packaged as Helm charts so the entire environment can be stood up reproducibly with a single helm install

Application Architecture & User Flows

- Frontend to backend: React UI communicates with a FastAPI server over REST; all API calls are authenticated using a bearer token passed in the auth-token header
- Bearer token auth flow: user submits username and password on the frontend → request goes to a dedicated auth container → auth container validates credentials against the user store and returns a short-lived bearer token → frontend stores the token in memory and attaches it to every subsequent API request
- New user KYC registration flow (async, event-driven):
 - User fills the registration form on the frontend
 - Form submission produces a KYC event to a Kafka topic
 - Verifier service consumes from the KYC topic; the verifier reviews the submission on their own landing page within the same website
 - On approval, the verifier publishes a message to the verified topic in Kafka
 - A downstream service reads from the verified topic and writes the new user record to the Users database
 - A welcome notification is enqueued to the notification queue → SHIM layer delivers a welcome message to the user
- Existing user login flow: authenticated user → RBAC check → queries alumni or fraternity data → response served from DynamoDB (authoritative) or Redis (cache layer for frequently-read data) depending on cache hit
- Internal admin flow: admin user authenticates with elevated role → can verify or deny pending KYC submissions, add new content to the alumni website, and create events visible to all members

LocalStack on Kubernetes & DynamoDB Backup

- LocalStack deployment on Kubernetes: LocalStack runs as a Kubernetes Deployment inside the cluster, exposed via a ClusterIP service; all other services in the mesh resolve it by DNS name and hit the LocalStack endpoint instead of real AWS
 - Helm chart for LocalStack: packages the Deployment, Service, ConfigMap (AWS region and service config), and PersistentVolumeClaim for data persistence cluster can be torn down and restarted without losing LocalStack state
-

- FastAPI REST wrapper: a FastAPI container wraps all LocalStack S3 and DynamoDB operations behind HTTP endpoints, other services interact with AWS resources via this REST API rather than calling the AWS SDK directly, enforcing a single access point
 - DynamoDB backup strategy from LocalStack:
 - Scheduled export: a Kubernetes CronJob runs periodically and calls the DynamoDB ExportTableToPointInTime API (supported by LocalStack) to export table data to S3
 - S3 versioning enabled on the backup bucket so each export is retained as a separate object version
 - Restore procedure: import the exported JSON back into DynamoDB using a Python script, validate record counts, and confirm table integrity
-